

Universidad Autónoma del Estado de México

Unidad Académica Profesional Tianguistenco

Ingeniería en software

Unidad de aprendizaje:

Cómputo Evolutivo

Profesor:

Jonathan Rojas Simón

Alumno:

Moisés Hernández Rodríguez

Fecha de entrega: 06/12/2022

Índice

Contenido

Introducción 3

Planteamiento del problema 4

Codificación del Individuo 4

Representación del individuo 4

Función de aptitud 5

Operadores usados 5

Cruce 5

Selección 6

Mutación 6

Criterios de parada 7

Pruebas de ejecución 8

Ejecución #1 8

Ejecución #2 11

Ejecución #3 12

Ejecución #4 13

Ejecución #5 14

Ejecución #6 15

Ejecución #7 16

Ejecución #8 17

Ejecución #9 18

Ejecución #10 19

Descripción de la solución 22

Conclusiones 23

Referencias 24

Introducción

Un algoritmo genético (AG) es una técnica de resolución de problemas que imita a la evolución biológica como estrategia para resolver problemas, englobándose dentro de lo que antes hemos denominado técnicas basadas en poblaciones. Dado un problema específico a resolver, la entrada del AG es un conjunto de soluciones potenciales a ese problema, codificadas de alguna manera, y una métrica llamada función de aptitud, o fitness, que permite evaluar cuantitativamente a cada solución candidata. Estas candidatas pueden ser soluciones que ya se sabe que funcionan, con el objetivo de que el AG las mejore, pero se suelen generar aleatoriamente.

A partir de ahí, AG evalúa cada candidata de acuerdo con la función de aptitud. Por supuesto, se debe tener en cuenta que estas primeras candidatas generadas aleatoriamente, tendrán una eficiencia mínima con respecto a la resolución del problema, y la mayoría no funcionarán en absoluto. Sin embargo, por puro azar, unas pocas pueden ser prometedoras, pudiendo mostrar algunas características que muestren, aunque sólo sea de una forma débil e imperfecta, cierta capacidad de solución del problema.

Estas candidatas prometedoras se conservan y se les permite reproducirse. Se realizan múltiples copias de ellas, pero estas copias no son perfectas, sino que se les introducen algunos cambios aleatorios durante el proceso de copia, a modo de las mutaciones que pueden sufrir los descendientes de una población. Luego, esta descendencia digital prosigue con la siguiente generación, formando un nuevo conjunto de soluciones candidatas, y son de nuevo sometidas a una ronda de evaluación de aptitud. Las candidatas que han empeorado o no han mejorado con los cambios en su código son eliminadas de nuevo; pero, de nuevo, por puro azar, las variaciones aleatorias introducidas en la población pueden haber mejorado a algunos individuos, convirtiéndolos en mejores soluciones del problema, más completas o eficientes. El proceso se repite las iteraciones que haga falta, hasta que obtengamos soluciones suficientemente buenas para nuestros propósitos.

Planteamiento del problema

Un sudoku es un tablero de 9x9 casillas en las cuales se deben colocar los números del 1 al 9, de manera que en cada fila y columna se encuentren todos ellos sin repetición. El tablero está dividido en 9 sectores de 3x3 casillas que también deben contener todos estos números. Como pista inicial, se proporcionan unas cuantas casillas fijas con parte de la solución final.

El Sudoku puede considerarse un problema de optimización con restricciones, a nivel de fila, columna y subcuadrícula.

Objetivo: Utilizar un algoritmo genético para resolver el sudoku.

Codificación del Individuo

Representación del individuo

El Sudoku es un claro ejemplo de representación basado en otros problemas de optimización combinatoria. Cada individuo (tablero) se codifica en un array de 81 posiciones; este array representa el cromosoma que está formado por 9 genes (filas). De esta forma, supongamos que tenemos el siguiente tablero.

		6						
	8			5	4	2		
	4			9			7	
		7	9			3		
				8		4		
6						1		
2		3						1
			5				4	
		8	3			5		2

Una representación recomendada es que cada individuo o cromosoma se representa por un array compuesto por 9 genes de 9 elementos cada uno. Cada gen se corresponde con una fila del tablero.

Las posiciones fijas del tablero se respetan y las posiciones vacías se representan como 0:

0	0	6	0	0	0	0	0	0

Cada valor del individuo o cromosoma tendrá un valor del 1 al 9. El cromosoma se inicializa con valores aleatorios entre 1 y 9, respetando las posiciones fijas y sin generar elementos repetidos dentro del gen (fila).

Función de aptitud

Función de aptitud: La función de aptitud se basa en que los elementos de cada fila y columna deben ser los valores del conjunto Dígitos = {1,2,3,4,5,6,7,8,9}. La función calcula el número de elementos que faltan en cada submatriz y columna. A nivel de fila no existe penalización pues los elementos son permutaciones.

Aptitud= Total de faltas de cada subcuadrícula + total de faltas de las columnas

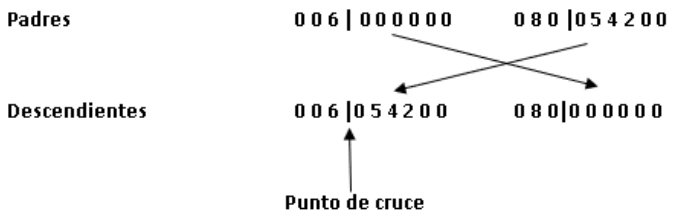
Se trata de un problema de minimización y el objetivo a conseguir aptitud 0.

Operadores usados

```
def data(ngeneraciones,cruza,mutacion):
    sqlite_adapter = DBAdapters.DBSQLite(identify="sudoku", resetDB=True)  #Para generar la grafica de aptitud
    genoma = G1DList.G1DList(81)
    genoma.setParams(rangemin=1,rangemax=9)
    genoma.evaluator.set(aptitud)
    genoma.setParams(bestrawscore=0.00, roundedecimal=2) # redondea las aptitudes
    genoma.crossover.set(Crossovers.G1DListCrossoverSinglePoint) # cambia la cruza a de un solo punto
    ga = GSimpleGA.GSimpleGA(genoma)
    ga.selector.set(Selectors.GRouletteWheel) # cambia la seleccion por ruleta
    ga.setMinimax(Consts.minimaxType["minimize"]) # se minimiza para reducir la cantidad de ceros del tablero
    ga.setMutationRate(mutacion) #asignamos el % de mutacion
    ga.setGenerations(ngeneraciones) #asignamos la cantidad de genereaciones a crear
    ga.setPopulationSize(81) #defiine el tamaño de la poblacion de nuestro tablero
    ga.setCrossoverRate(cruza) # definimos el rango de cruza
    ga.setElitismReplacement(1) # para eligir un solo individuo elite
    ga.terminationCriteria.set(GSimpleGA.RawScoreCriteria)
    ga.setDBAdapter(sqlite_adapter) #definimos donde se guardaran los datos
    ga.evolve(freq_stats=100) #definimos la cantidad de frequencia a la que evolucionara
    best = ga.bestIndividual() # generamos el individuo mas apto
    print best
```

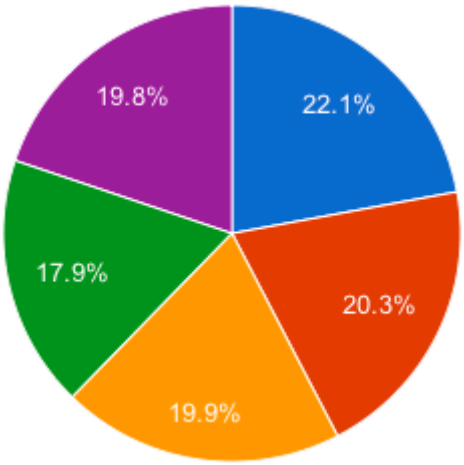
Cruce

Cruce basado en un punto: los dos individuos seleccionados para jugar el papel de padres son recombinados por medio de la selección de un punto de corte, para posteriormente intercambiar las secciones que se encuentran a la derecha de dicho punto. Es decir, los genes del padre1 a la izquierda del punto de corte forman parte del hijo1 y los situados a la derecha formaran parte del hijo2, mientras que con el padre2 sucederá lo contrario. Se escoge un punto de cruce divisorio entre filas y se intercambian los cuadros de los padres hasta ese punto.



Selección

Selección por ruleta: A cada uno de los individuos de la población se le asigna una parte proporcional a su ajuste de una ruleta, de tal forma que la suma de todos los porcentajes sea la unidad. Los mejores individuos recibirán una porción de la ruleta mayor que la recibida por los peores. Generalmente la población está ordenada en base al ajuste por lo que las porciones más grandes se encuentran al inicio de la ruleta. Para seleccionar un individuo basta con generar un número aleatorio del intervalo [0 - 1] y devolver el individuo situado en esa posición de la ruleta. Esta posición se suele obtener recorriendo los individuos de la población y acumulando sus proporciones de ruleta hasta que la suma exceda el valor obtenido.



Mutación

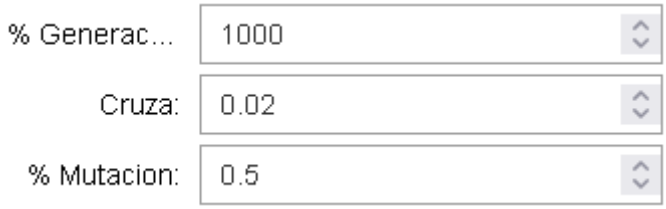
Mutación por intercambio

1. Se selecciona probabilísticamente el (los) gen(es) candidatos.
2. Se selecciona aleatoriamente una posición del mismo individuo.
3. Se realiza intercambio de genes.

El rango de mutación debe ser >= 0.0 y <= 1.0 (mayor a cero y meno que uno)

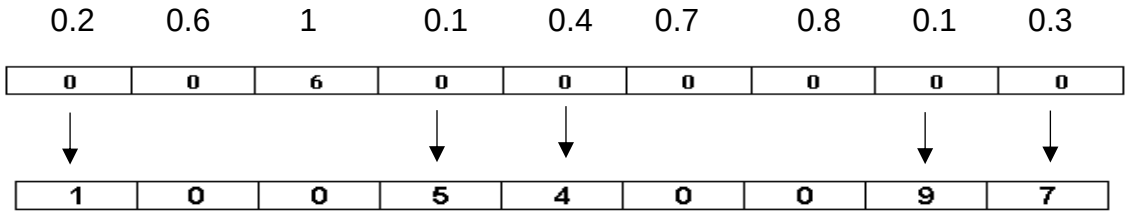
Donde pm = es el % de mutación establecido

Ejemplo:

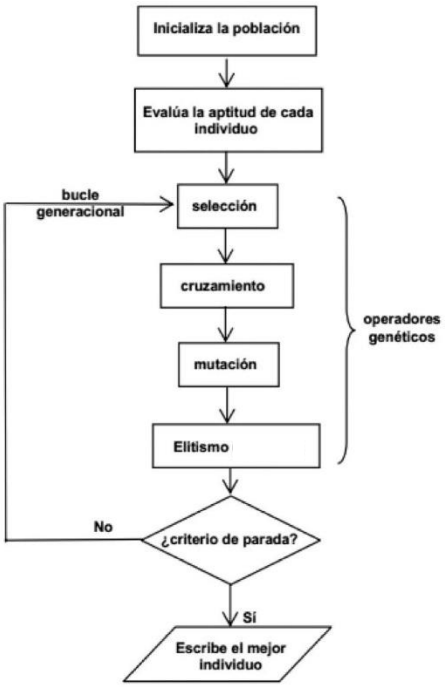


Mutación = 0.5 = pm

Se hace un tiro de aleatorio por cada campo del arreglo donde si el valor es 0.0 a 0.5 se realiza la mutación eligiendo un valor aleatorio entre el 1 y el 9 que es nuestro rango y donde 0.5 al 1.0 no se realiza la mutación.



Se utiliza elitismo para determinar valores óptimos.



El valor de la población es igual a 81 debido al tamaño del sudoku y este valor no debe variar para obtener resultados óptimos.

Se toma el individuo mas apto y con menor índice de aptitud para mostrar.

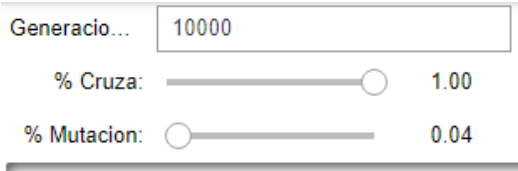
Criterios de parada

Para que el programa pueda terminar su ejecución de manera satisfactoria se deben cumplir los siguientes criterios:

1. Cumplir con el número de generaciones establecidas por el usuario.
2. Llegar al valor de aptitud que es igual a 0 ya que el problema es de minimización.

Pruebas de ejecución

Ejecución #1




```
Gen. 0 (0.00%): Max/Min/Avg Fitness(Raw) [170.07(159.00)/114.27(125.00)/141.73(141.73)]
Gen. 100 (1.00%): Max/Min/Avg Fitness(Raw) [151.64(143.00)/84.77(99.00)/126.37(126.37)]
Gen. 200 (2.00%): Max/Min/Avg Fitness(Raw) [154.96(145.00)/71.93(94.00)/129.14(129.14)]
Gen. 300 (3.00%): Max/Min/Avg Fitness(Raw) [152.89(143.00)/69.54(92.00)/127.41(127.41)]
Gen. 400 (4.00%): Max/Min/Avg Fitness(Raw) [144.70(137.00)/75.67(90.00)/120.58(120.58)]
Gen. 500 (5.00%): Max/Min/Avg Fitness(Raw) [151.90(139.00)/43.86(86.00)/126.58(126.58)]
Gen. 600 (6.00%): Max/Min/Avg Fitness(Raw) [145.13(146.00)/87.22(86.00)/120.94(120.94)]
Gen. 700 (7.00%): Max/Min/Avg Fitness(Raw) [147.48(139.00)/66.56(86.00)/122.90(122.90)]
Gen. 800 (8.00%): Max/Min/Avg Fitness(Raw) [147.59(140.00)/69.51(86.00)/122.99(122.99)]
Gen. 900 (9.00%): Max/Min/Avg Fitness(Raw) [149.38(139.00)/58.49(86.00)/124.48(124.48)]
Gen. 1000 (10.00%): Max/Min/Avg Fitness(Raw) [150.87(141.00)/60.31(86.00)/125.73(125.73)]
Gen. 1100 (11.00%): Max/Min/Avg Fitness(Raw) [153.05(145.00)/66.84(86.00)/127.54(127.54)]
Gen. 1200 (12.00%): Max/Min/Avg Fitness(Raw) [149.16(142.00)/70.52(86.00)/124.30(124.30)]
Gen. 1300 (13.00%): Max/Min/Avg Fitness(Raw) [151.97(141.00)/54.95(86.00)/126.64(126.64)]
Gen. 1400 (14.00%): Max/Min/Avg Fitness(Raw) [151.05(141.00)/59.50(86.00)/125.88(125.88)]
Gen. 1500 (15.00%): Max/Min/Avg Fitness(Raw) [154.07(145.00)/62.83(86.00)/128.40(128.40)]
Gen. 1600 (16.00%): Max/Min/Avg Fitness(Raw) [150.87(144.00)/69.68(85.00)/125.73(125.73)]
Gen. 1700 (17.00%): Max/Min/Avg Fitness(Raw) [146.68(144.00)/80.41(85.00)/122.23(122.23)]
Gen. 1800 (18.00%): Max/Min/Avg Fitness(Raw) [146.15(144.00)/80.34(84.00)/121.79(121.79)]
Gen. 1900 (19.00%): Max/Min/Avg Fitness(Raw) [145.32(135.00)/47.75(79.00)/121.10(121.10)]
Gen. 2000 (20.00%): Max/Min/Avg Fitness(Raw) [140.92(138.00)/72.40(78.00)/117.43(117.43)]
Gen. 2100 (21.00%): Max/Min/Avg Fitness(Raw) [144.95(143.00)/74.25(78.00)/120.79(120.79)]
Gen. 2200 (22.00%): Max/Min/Avg Fitness(Raw) [147.33(144.00)/70.97(78.00)/122.78(122.78)]
Gen. 2300 (23.00%): Max/Min/Avg Fitness(Raw) [144.58(136.00)/54.52(78.00)/120.48(120.48)]
Gen. 2400 (24.00%): Max/Min/Avg Fitness(Raw) [147.14(144.00)/71.45(78.00)/122.62(122.62)]
Gen. 2500 (25.00%): Max/Min/Avg Fitness(Raw) [144.73(141.00)/70.22(78.00)/120.60(120.60)]
Gen. 2600 (26.00%): Max/Min/Avg Fitness(Raw) [147.76(141.00)/59.53(77.00)/123.14(123.14)]
Gen. 2700 (27.00%): Max/Min/Avg Fitness(Raw) [144.15(139.00)/61.42(74.00)/120.12(120.12)]
Gen. 2800 (28.00%): Max/Min/Avg Fitness(Raw) [143.91(140.00)/65.05(74.00)/119.93(119.93)]
Gen. 2900 (29.00%): Max/Min/Avg Fitness(Raw) [145.51(141.00)/63.20(74.00)/121.26(121.26)]
Gen. 3000 (30.00%): Max/Min/Avg Fitness(Raw) [139.54(138.00)/69.93(73.00)/116.28(116.28)]
Gen. 3100 (31.00%): Max/Min/Avg Fitness(Raw) [137.30(138.00)/73.25(72.00)/114.42(114.42)]
Gen. 3200 (32.00%): Max/Min/Avg Fitness(Raw) [137.78(136.00)/68.41(72.00)/114.81(114.81)]
Gen. 3300 (33.00%): Max/Min/Avg Fitness(Raw) [136.77(132.00)/60.89(72.00)/113.98(113.98)]
Gen. 3400 (34.00%): Max/Min/Avg Fitness(Raw) [138.01(133.00)/60.01(72.00)/115.01(115.01)]
Gen. 3500 (35.00%): Max/Min/Avg Fitness(Raw) [144.46(143.00)/65.68(69.00)/120.38(120.38)]
Gen. 3600 (36.00%): Max/Min/Avg Fitness(Raw) [136.18(136.00)/68.65(69.00)/113.48(113.48)]
Gen. 3700 (37.00%): Max/Min/Avg Fitness(Raw) [140.18(137.00)/61.47(69.00)/116.81(116.81)]
Gen. 3800 (38.00%): Max/Min/Avg Fitness(Raw) [140.04(138.00)/64.42(69.00)/116.70(116.70)]
Gen. 3900 (39.00%): Max/Min/Avg Fitness(Raw) [140.43(141.00)/70.14(69.00)/117.02(117.02)]
Gen. 4000 (40.00%): Max/Min/Avg Fitness(Raw) [141.57(138.00)/60.27(69.00)/117.98(117.98)]
Gen. 4100 (41.00%): Max/Min/Avg Fitness(Raw) [139.21(135.00)/58.56(69.00)/116.01(116.01)]
Gen. 4200 (42.00%): Max/Min/Avg Fitness(Raw) [134.90(132.00)/62.56(69.00)/112.42(112.42)]
Gen. 4300 (43.00%): Max/Min/Avg Fitness(Raw) [144.59(141.00)/59.98(69.00)/120.49(120.49)]
Gen. 4400 (44.00%): Max/Min/Avg Fitness(Raw) [145.70(140.00)/52.91(69.00)/121.42(121.42)]
Gen. 4500 (45.00%): Max/Min/Avg Fitness(Raw) [143.90(141.00)/62.01(69.00)/119.91(119.91)]
Gen. 4600 (46.00%): Max/Min/Avg Fitness(Raw) [139.59(135.00)/57.38(69.00)/116.32(116.32)]
Gen. 4700 (47.00%): Max/Min/Avg Fitness(Raw) [141.05(138.00)/60.61(68.00)/117.54(117.54)]
```

```
Gen. 4800 (48.00%): Max/Min/Avg Fitness(Raw) [142.39(138.00)/56.52(68.00)/118.65(118.65)]
Gen. 4900 (49.00%): Max/Min/Avg Fitness(Raw) [139.45(136.00)/59.59(68.00)/116.21(116.21)]
Gen. 5000 (50.00%): Max/Min/Avg Fitness(Raw) [135.21(132.00)/60.57(68.00)/112.68(112.68)]
Gen. 5100 (51.00%): Max/Min/Avg Fitness(Raw) [138.76(135.00)/58.77(68.00)/115.63(115.63)]
Gen. 5200 (52.00%): Max/Min/Avg Fitness(Raw) [143.81(137.00)/47.44(68.00)/119.84(119.84)]
Gen. 5300 (53.00%): Max/Min/Avg Fitness(Raw) [141.94(138.00)/57.95(68.00)/118.28(118.28)]
Gen. 5400 (54.00%): Max/Min/Avg Fitness(Raw) [142.43(139.00)/59.44(68.00)/118.69(118.69)]
Gen. 5500 (55.00%): Max/Min/Avg Fitness(Raw) [137.21(134.00)/60.42(68.00)/114.35(114.35)]
Gen. 5600 (56.00%): Max/Min/Avg Fitness(Raw) [141.76(141.00)/66.33(68.00)/118.14(118.14)]
Gen. 5700 (57.00%): Max/Min/Avg Fitness(Raw) [140.73(137.00)/57.51(67.00)/117.27(117.27)]
Gen. 5800 (58.00%): Max/Min/Avg Fitness(Raw) [139.54(139.00)/65.83(67.00)/116.28(116.28)]
Gen. 5900 (59.00%): Max/Min/Avg Fitness(Raw) [140.12(137.00)/59.33(67.00)/116.77(116.77)]
Gen. 6000 (60.00%): Max/Min/Avg Fitness(Raw) [131.99(135.00)/72.18(67.00)/109.99(109.99)]
Gen. 6100 (61.00%): Max/Min/Avg Fitness(Raw) [136.24(140.00)/73.62(67.00)/113.53(113.53)]
Gen. 6200 (62.00%): Max/Min/Avg Fitness(Raw) [144.31(139.00)/51.91(67.00)/120.26(120.26)]
Gen. 6300 (63.00%): Max/Min/Avg Fitness(Raw) [139.66(139.00)/65.56(67.00)/116.38(116.38)]
Gen. 6400 (64.00%): Max/Min/Avg Fitness(Raw) [135.69(135.00)/64.52(66.00)/113.07(113.07)]
Gen. 6500 (65.00%): Max/Min/Avg Fitness(Raw) [143.70(139.00)/52.86(66.00)/119.75(119.75)]
Gen. 6600 (66.00%): Max/Min/Avg Fitness(Raw) [139.99(143.00)/71.80(66.00)/116.65(116.65)]
Gen. 6700 (67.00%): Max/Min/Avg Fitness(Raw) [139.99(135.00)/50.96(65.00)/116.65(116.65)]
Gen. 6800 (68.00%): Max/Min/Avg Fitness(Raw) [136.70(136.00)/63.46(65.00)/113.91(113.91)]
Gen. 6900 (69.00%): Max/Min/Avg Fitness(Raw) [130.62(127.00)/53.85(63.00)/108.85(108.85)]
Gen. 7000 (70.00%): Max/Min/Avg Fitness(Raw) [133.32(128.00)/47.86(63.00)/111.10(111.10)]
Gen. 7100 (71.00%): Max/Min/Avg Fitness(Raw) [133.41(130.00)/54.28(63.00)/111.17(111.17)]
Gen. 7200 (72.00%): Max/Min/Avg Fitness(Raw) [139.39(134.00)/45.63(62.00)/116.16(116.16)]
Gen. 7300 (73.00%): Max/Min/Avg Fitness(Raw) [138.80(133.00)/44.04(62.00)/115.67(115.67)]
Gen. 7400 (74.00%): Max/Min/Avg Fitness(Raw) [142.99(138.00)/44.32(60.00)/119.16(119.16)]
Gen. 7500 (75.00%): Max/Min/Avg Fitness(Raw) [138.98(135.00)/48.43(60.00)/115.81(115.81)]
Gen. 7600 (76.00%): Max/Min/Avg Fitness(Raw) [136.40(135.00)/56.48(60.00)/113.67(113.67)]
Gen. 7700 (77.00%): Max/Min/Avg Fitness(Raw) [134.92(135.00)/60.19(60.00)/112.43(112.43)]
Gen. 7800 (78.00%): Max/Min/Avg Fitness(Raw) [139.17(135.00)/47.73(60.00)/115.98(115.98)]
Gen. 7900 (79.00%): Max/Min/Avg Fitness(Raw) [135.85(133.00)/52.33(60.00)/113.21(113.21)]
Gen. 8000 (80.00%): Max/Min/Avg Fitness(Raw) [134.00(133.00)/57.58(60.00)/111.67(111.67)]
Gen. 8100 (81.00%): Max/Min/Avg Fitness(Raw) [143.04(139.00)/47.93(60.00)/119.20(119.20)]
Gen. 8200 (82.00%): Max/Min/Avg Fitness(Raw) [137.60(134.00)/49.82(60.00)/114.67(114.67)]
Gen. 8300 (83.00%): Max/Min/Avg Fitness(Raw) [132.33(130.00)/54.07(60.00)/110.27(110.27)]
Gen. 8400 (84.00%): Max/Min/Avg Fitness(Raw) [141.29(139.00)/53.78(60.00)/117.74(117.74)]
Gen. 8500 (85.00%): Max/Min/Avg Fitness(Raw) [135.69(138.00)/64.92(60.00)/113.07(113.07)]
Gen. 8600 (86.00%): Max/Min/Avg Fitness(Raw) [136.81(140.00)/66.62(60.00)/114.01(114.01)]
Gen. 8700 (87.00%): Max/Min/Avg Fitness(Raw) [144.39(143.00)/56.32(60.00)/120.32(120.32)]
Gen. 8800 (88.00%): Max/Min/Avg Fitness(Raw) [135.97(134.00)/54.92(60.00)/113.31(113.31)]
Gen. 8900 (89.00%): Max/Min/Avg Fitness(Raw) [136.73(134.00)/52.67(60.00)/113.94(113.94)]
Gen. 9000 (90.00%): Max/Min/Avg Fitness(Raw) [136.70(135.00)/55.66(60.00)/113.91(113.91)]
Gen. 9100 (91.00%): Max/Min/Avg Fitness(Raw) [138.65(139.00)/60.82(60.00)/115.54(115.54)]

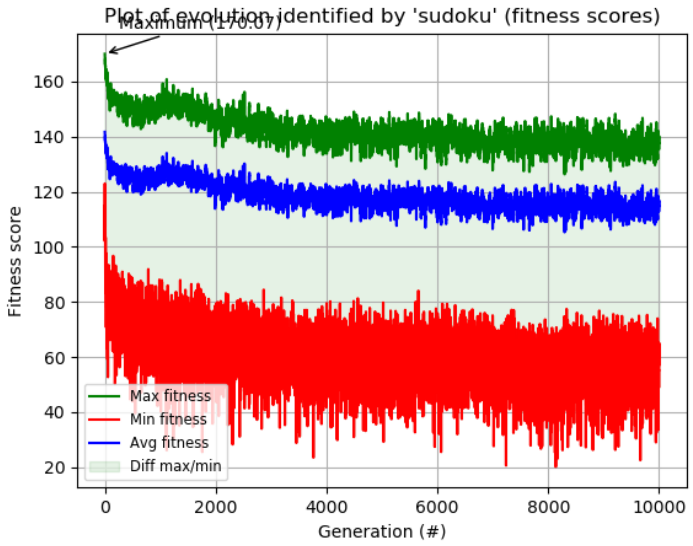
Gen. 9200 (92.00%): Max/Min/Avg Fitness(Raw) [137.33(136.00)/55.57(59.00)/114.44(114.44)]
Gen. 9300 (93.00%): Max/Min/Avg Fitness(Raw) [132.68(135.00)/63.89(59.00)/110.57(110.57)]
Gen. 9400 (94.00%): Max/Min/Avg Fitness(Raw) [137.16(132.00)/42.90(59.00)/114.30(114.30)]
Gen. 9500 (95.00%): Max/Min/Avg Fitness(Raw) [138.93(135.00)/47.38(59.00)/115.78(115.78)]
Gen. 9600 (96.00%): Max/Min/Avg Fitness(Raw) [135.69(138.00)/64.01(59.00)/113.07(113.07)]
Gen. 9700 (97.00%): Max/Min/Avg Fitness(Raw) [146.81(142.00)/43.48(59.00)/122.35(122.35)]
Gen. 9800 (98.00%): Max/Min/Avg Fitness(Raw) [140.06(140.00)/58.85(59.00)/116.72(116.72)]
Gen. 9900 (99.00%): Max/Min/Avg Fitness(Raw) [133.97(138.00)/67.05(59.00)/111.64(111.64)]
Gen. 10000 (100.00%): Max/Min/Avg Fitness(Raw) [139.56(142.00)/64.45(59.00)/116.30(116.30)]
Total time elapsed: 399.961 seconds.
```

```
- GenomeBase
        Score:                  59.000000
        Fitness:                64.448927

        Params:          {'rounddecimal': 2, 'bestrawscore': 0.0, 'rangemax': 9, 'rangemin': 1}

        Slot [Evaluator] (Count: 1)
                Name: aptitud - Weight: 0.50
        Slot [Initializator] (Count: 1)
                Name: G1DListInitializatorInteger - Weight: 0.50
                Doc:  Integer initialization function of G1DList

   This initializator accepts the *rangemin* and *rangemax* genome parameters.
```



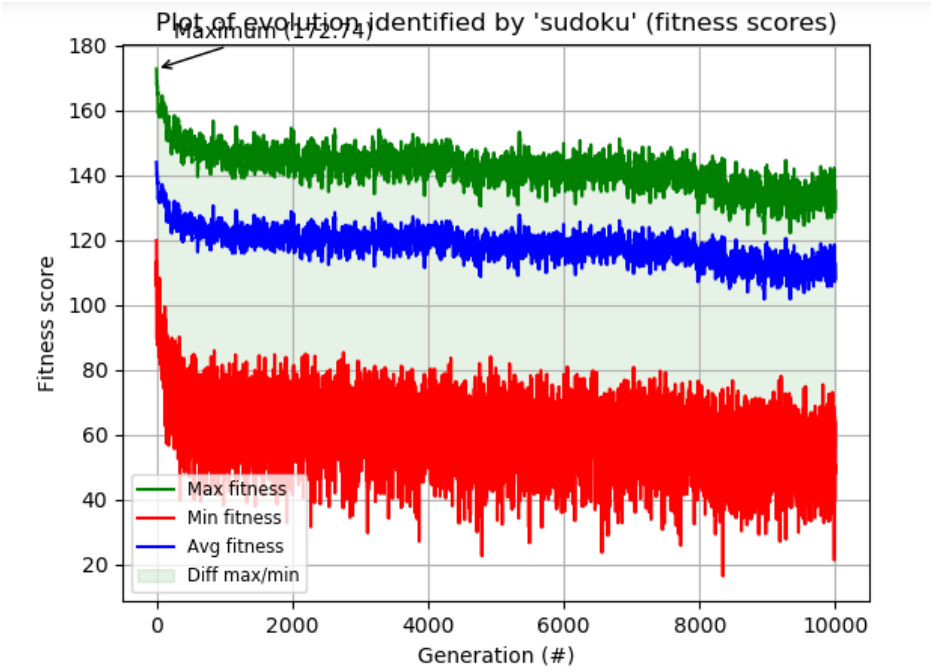
Ejecución #2

```
Gen. 10000 (100.00%): Max/Min/Avg Fitness(Raw) [129.85(135.00)/65.23(55.00)/108.21(108.21)]
Total time elapsed: 418.334 seconds.
- GenomeBase
        Score:                  55.000000
        Fitness:                65.225124

        Params:          {'rounddecimal': 2, 'bestrawscore': 0.0, 'rangemax': 9, 'rangemin': 1}

        Slot [Evaluator] (Count: 1)
                Name: aptitud - Weight: 0.50
        Slot [Initializator] (Count: 1)
                Name: G1DListInitializatorInteger - Weight: 0.50
                Doc:  Integer initialization function of G1DList

   This initializator accepts the *rangemin* and *rangemax* genome parameters.
```



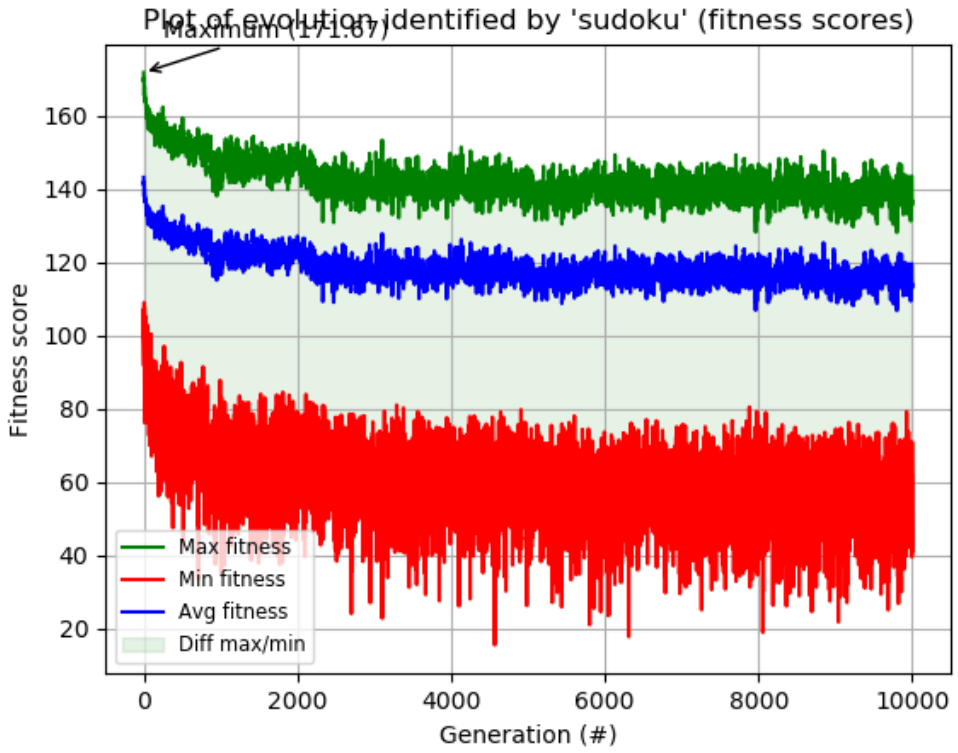
Ejecución #3

```
Gen. 10000 (100.00%): Max/Min/Avg Fitness(Raw) [137.76(134.00)/51.65(62.00)/114.80(114.80)]
Total time elapsed: 423.768 seconds.
- GenomeBase
        Score:                  62.000000
        Fitness:                51.650037

        Params:          {'rounddecimal': 2, 'bestrawscore': 0.0, 'rangemax': 9, 'rangemin': 1}

        Slot [Evaluator] (Count: 1)
                Name: aptitud - Weight: 0.50
        Slot [Initializator] (Count: 1)
                Name: G1DListInitializatorInteger - Weight: 0.50
                Doc:  Integer initialization function of G1DList

   This initializator accepts the *rangemin* and *rangemax* genome parameters.
```

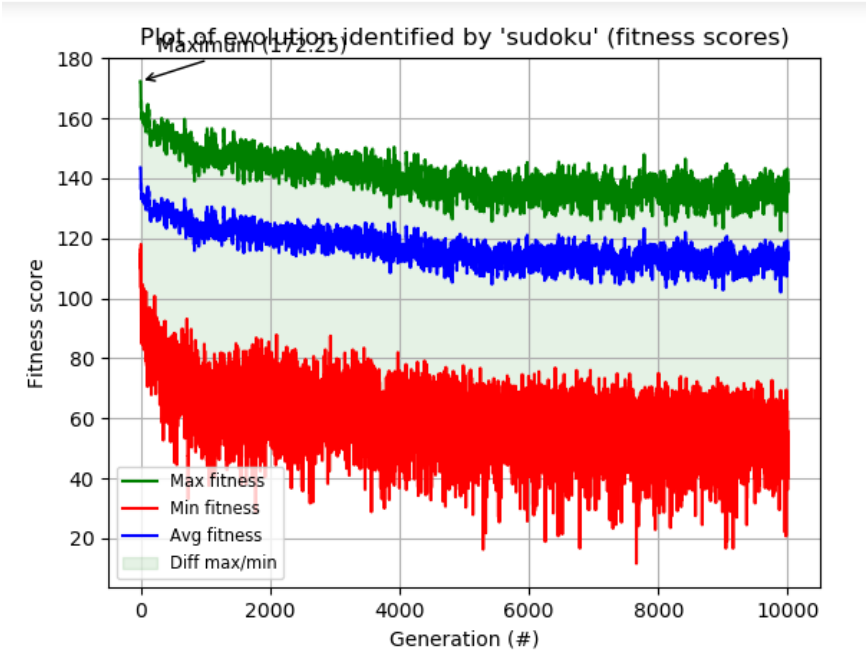


Ejecución #4

```
Total time elapsed: 392.736 seconds.
- GenomeBase
        Score:                  54.000000
        Fitness:                47.999389

        Params:          {'rounddecimal': 2, 'bestrawscore': 0.0, 'rangemax': 9, 'rangemin': 1}

        Slot [Evaluator] (Count: 1)
                Name: aptitud - Weight: 0.50
        Slot [Initializator] (Count: 1)
                Name: G1DListInitializatorInteger - Weight: 0.50
                Doc:  Integer initialization function of G1DList
```



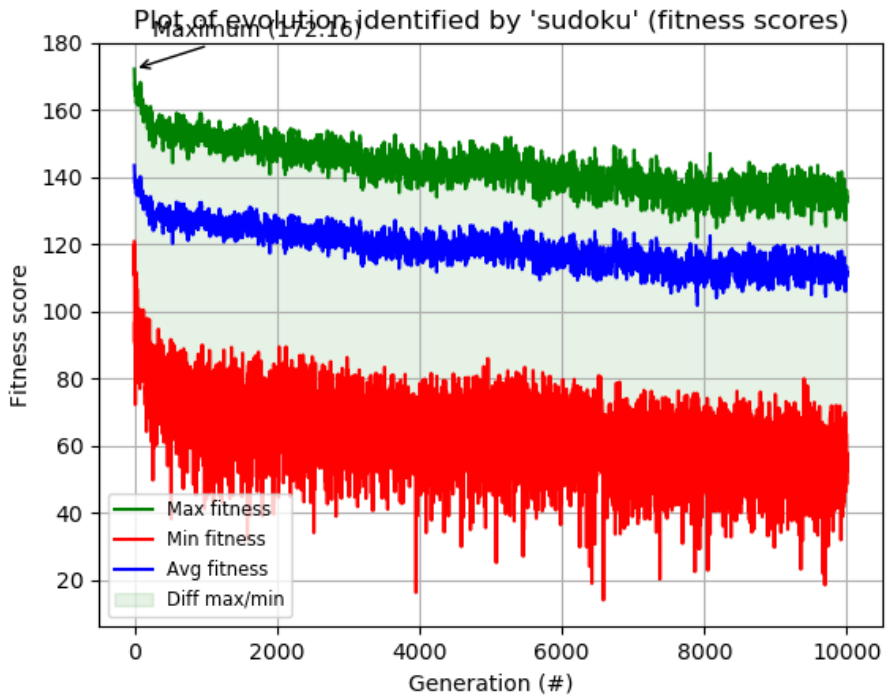
Ejecución #5

```
Gen. 10000 (100.00%): Max/Min/Avg Fitness(Raw) [136.06(131.00)/40.81(57.00)/113.38(113.38)]
Total time elapsed: 485.543 seconds.
- GenomeBase
        Score:                  57.000000
        Fitness:                40.808243

        Params:          {'rounddecimal': 2, 'bestrawscore': 0.0, 'rangemax': 9, 'rangemin': 1}

        Slot [Evaluator] (Count: 1)
                Name: aptitud - Weight: 0.50
        Slot [Initializator] (Count: 1)
                Name: G1DListInitializatorInteger - Weight: 0.50
                Doc:  Integer initialization function of G1DList

   This initializator accepts the *rangemin* and *rangemax* genome parameters.
```



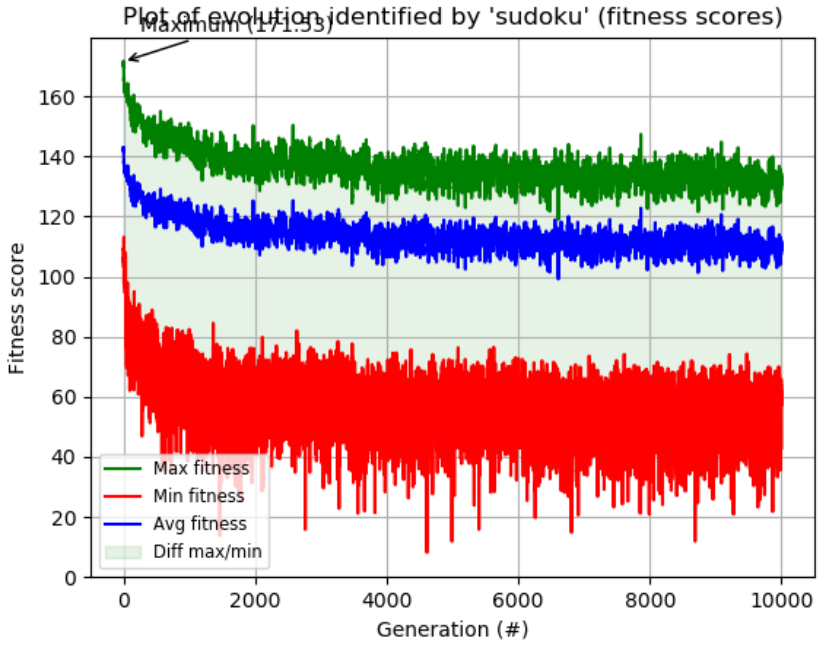
Ejecución #6

```
Gen. 10000 (100.00%): Max/Min/Avg Fitness(Raw) [127.54(129.00)/57.36(54.00)/106.28(106.28)]
Total time elapsed: 348.547 seconds.
- GenomeBase
        Score:                  54.000000
        Fitness:                57.358676

        Params:         {'rounddecimal': 2, 'bestrawscore': 0.0, 'rangemax': 9, 'rangemin': 1}

        Slot [Evaluator] (Count: 1)
                Name: aptitud - Weight: 0.50
        Slot [Initializator] (Count: 1)
                Name: G1DListInitializatorInteger - Weight: 0.50
                Doc:  Integer initialization function of G1DList

   This initializator accepts the *rangemin* and *rangemax* genome parameters.
```



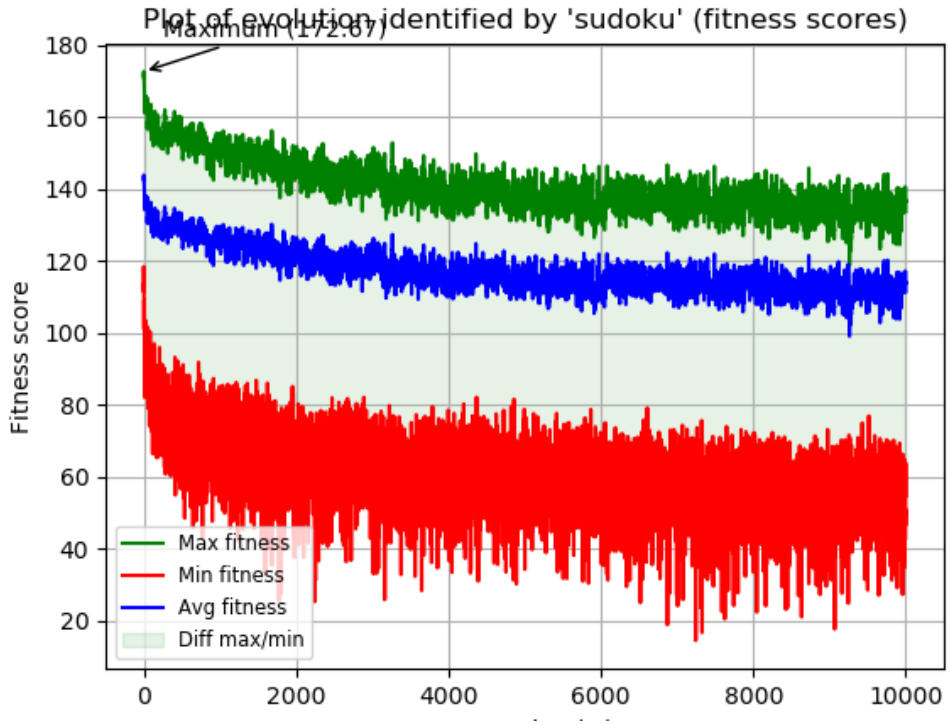
Ejecución #7

```
Gen. 10000 (100.00%): Max/Min/Avg Fitness(Raw) [134.46(134.00)/52.79(54.00)/112.05(112.05)]
Total time elapsed: 333.638 seconds.
- GenomeBase
        Score:                  54.000000
        Fitness:                52.785468

        Params:         {'rounddecimal': 2, 'bestrawscore': 0.0, 'rangemax': 9, 'rangemin': 1}

        Slot [Evaluator] (Count: 1)
                Name: aptitud - Weight: 0.50
        Slot [Initializator] (Count: 1)
                Name: G1DListInitializatorInteger - Weight: 0.50
                Doc:  Integer initialization function of G1DList

   This initializator accepts the *rangemin* and *rangemax* genome parameters.
```



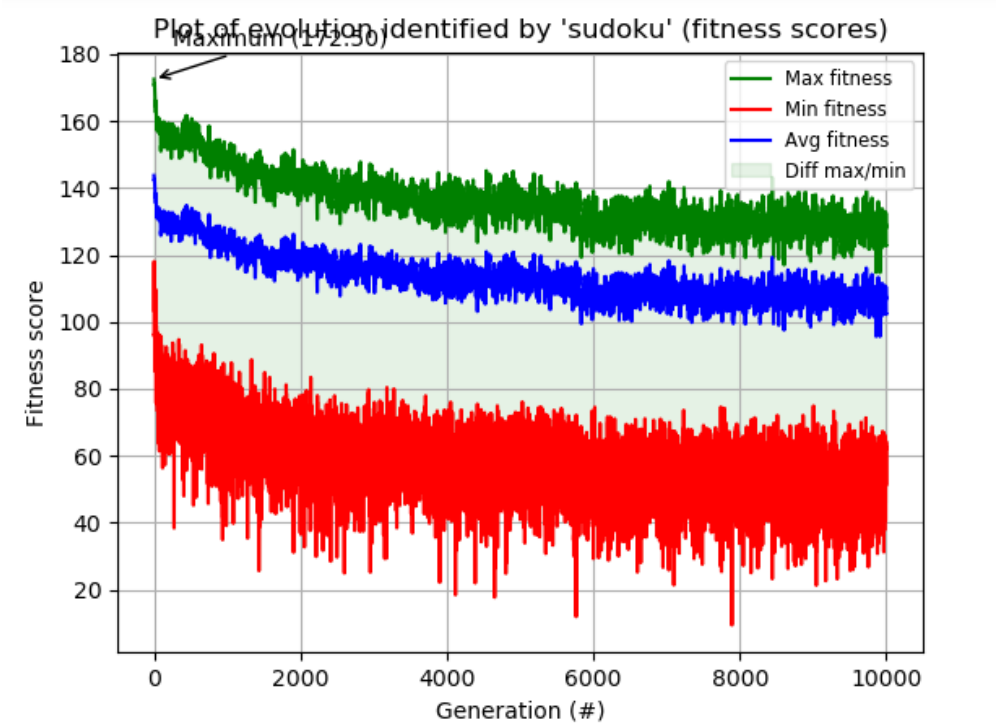
Ejecución #8

```
Gen. 10000 (100.00%): Max/Min/Avg Fitness(Raw) [123.16(133.00)/64.36(46.00)/102.63(102.63)]
Total time elapsed: 371.555 seconds.
- GenomeBase
        Score:                  46.000000
        Fitness:                64.356287

        Params:          {'rounddecimal': 2, 'bestrawscore': 0.0, 'rangemax': 9, 'rangemin': 1}

        Slot [Evaluator] (Count: 1)
                Name: aptitud - Weight: 0.50
        Slot [Initializator] (Count: 1)
                Name: G1DListInitializatorInteger - Weight: 0.50
                Doc:  Integer initialization function of G1DList

   This initializator accepts the *rangemin* and *rangemax* genome parameters.
```

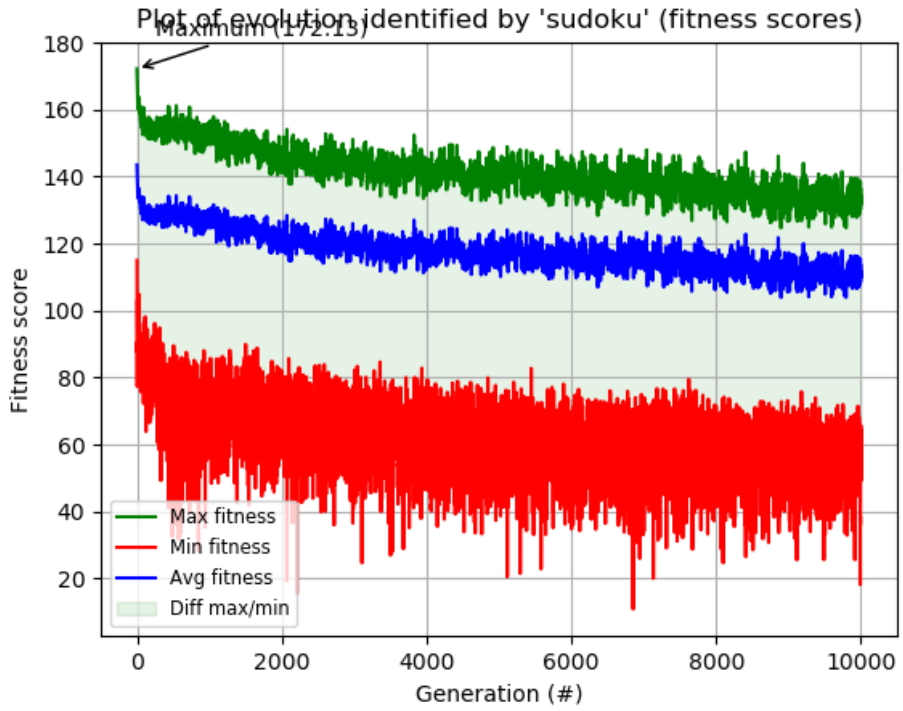


Ejecución #9

```
Gen. 10000 (100.00%): Max/Min/Avg Fitness(Raw) [128.87(130.00)/57.61(55.00)/107.40(107.40)]
Total time elapsed: 421.819 seconds.
- GenomeBase
        Score:                  55.000000
        Fitness:                57.609738

        Params:          {'rounddecimal': 2, 'bestrawscore': 0.0, 'rangemax': 9, 'rangemin': 1}

        Slot [Evaluator] (Count: 1)
                Name: aptitud - Weight: 0.50
        Slot [Initializator] (Count: 1)
                Name: G1DListInitializatorInteger - Weight: 0.50
                Doc:  Integer initialization function of G1DList
```

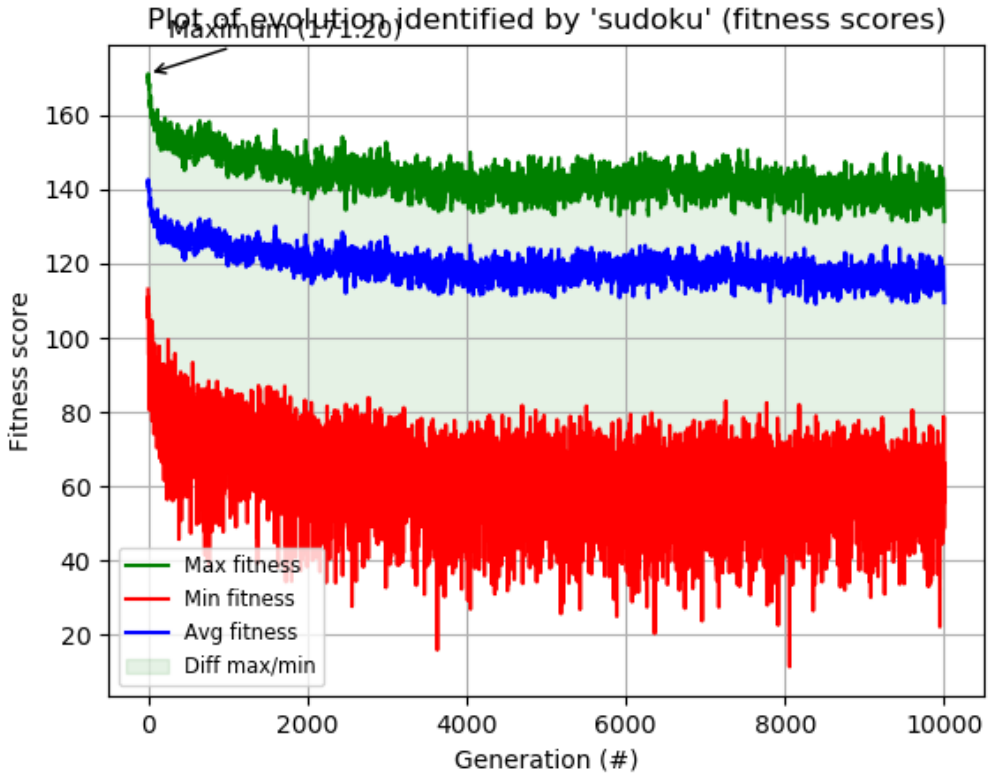


Ejecución #10

```
Gen. 10000 (100.00%): Max/Min/Avg Fitness(Raw) [131.94(132.00)/64.12(64.00)/109.95(109.95)]
Total time elapsed: 355.313 seconds.
- GenomeBase
        Score:                  64.000000
        Fitness:                64.123495

        Params:          {'rounddecimal': 2, 'bestrawscore': 0.0, 'rangemax': 9, 'rangemin': 1}

        Slot [Evaluator] (Count: 1)
                Name: aptitud - Weight: 0.50
        Slot [Initializator] (Count: 1)
                Name: G1DListInitializatorInteger - Weight: 0.50
                Doc:  Integer initialization function of G1DList
```



Corrida 1 millón de generaciones:

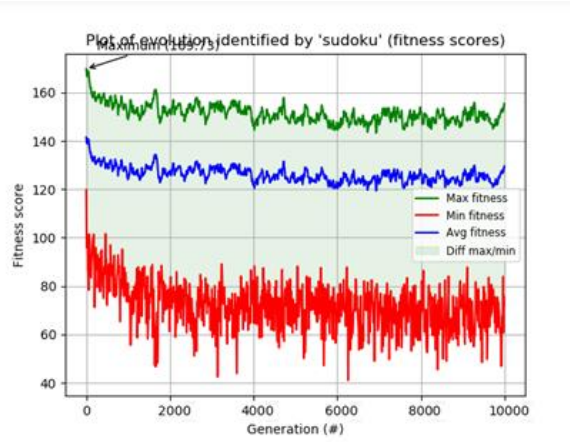
```
Gen. 999100 (99.91%): Max/Min/Avg Fitness(Raw) [110.18(123.00)/45.41(13.00)/91.81(91.81)]
Gen. 999200 (99.92%): Max/Min/Avg Fitness(Raw) [105.81(122.00)/48.98(13.00)/88.17(88.17)]
Gen. 999300 (99.93%): Max/Min/Avg Fitness(Raw) [114.28(122.00)/36.71(13.00)/95.23(95.23)]
Gen. 999400 (99.94%): Max/Min/Avg Fitness(Raw) [103.90(116.00)/43.27(13.00)/86.58(86.58)]
Gen. 999500 (99.95%): Max/Min/Avg Fitness(Raw) [114.96(124.00)/39.54(13.00)/95.80(95.80)]
Gen. 999600 (99.96%): Max/Min/Avg Fitness(Raw) [108.00(116.00)/36.69(13.00)/90.00(90.00)]
Gen. 999700 (99.97%): Max/Min/Avg Fitness(Raw) [107.29(125.00)/51.02(13.00)/89.41(89.41)]
Gen. 999800 (99.98%): Max/Min/Avg Fitness(Raw) [103.23(124.00)/52.94(13.00)/86.02(86.02)]
Gen. 999900 (99.99%): Max/Min/Avg Fitness(Raw) [120.29(117.00)/36.90(13.00)/95.99(95.99)]
Gen. 1000000 (100.00%): Max/Min/Avg Fitness(Raw) [109.57(118.00)/37.73(13.00)/91.31(91.31)]
Total time elapsed: 30344.177 seconds.
- GenomeBase
        Score:                  13.000000
        Fitness:                37.731332
```

```
        Slot [Crossover] (Count: 1)
                Name: G1DListCrossoverSinglePoint - Weight: 0.50
                Doc:  The crossover of G1DList, Single Point

   .. warning:: You can't use this crossover method for lists with just one element.


- G1DList
        List size:       81
        List:            [1, 2, 6, 8, 7, 3, 6, 9, 5, 7, 8, 9, 1, 5, 4, 2, 3, 6, 3, 4, 5, 2, 9, 6, 1, 7, 8, 9, 1, 7, 9, 2,
8, 3, 5, 6, 5, 6, 4, 3, 8, 1, 4, 2, 7, 6, 3, 2, 7, 4, 5, 1, 8, 9, 2, 5, 3, 4, 6, 7, 9, 8, 1, 8, 9, 1, 5, 3, 2, 7, 4, 3,
4, 7, 8, 3, 1, 9, 5, 6, 2]
```

Out[10]:




```
[1, 2, 6, 8, 7, 3, 6, 9, 5,
 7, 8, 9, 1, 5, 4, 2, 3, 6,
 3, 4, 5, 2, 9, 6, 1, 7, 8,

 9, 1, 7, 9, 2, 8, 3, 5, 6,
 5, 6, 4, 3, 8, 1, 4, 2, 7,
 6, 3, 2, 7, 4, 5, 1, 8, 9,

 2, 5, 3, 4, 6, 7, 9, 8, 1,
 8, 9, 1, 5, 3, 2, 7, 4, 3,
 4, 7, 8, 3, 1, 9, 5, 6, 2
```

Corrida 2 millones de generación:

```
Gen. 1999700 (99.98%):  Max/Min/Avg Fitness(Raw) [19.68(23.00)/11.69(0.00)/15.77(14.73)]
Gen. 1999800 (99.99%):  Max/Min/Avg Fitness(Raw) [19.01(23.00)/10.07(0.00)/15.27(14.73)]
Gen. 1999900 (100.00%): Max/Min/Avg Fitness(Raw) [18.85(23.00)/10.14(0.00)/14.98(14.73)]
Gen. 2000000 (100.00%): Max/Min/Avg Fitness(Raw) [17.68(23.00)/9.48(0.00)/14.73(14.73)]
Total time elapsed: 60259.396 seconds.
- GenomeBase
        Score:                  0.00
        Fitness:                9.48

        Params:          {'rounddecimal': 2, 'bestrawscore': 0.0, 'rangemax': 9, 'rangemin': 1}

        Slot [Evaluator] (Count: 1)
                Name: aptitud - Weight: 0.50
        Slot [Initializator] (Count: 1)
                Name: G1DListInitializatorInteger - Weight: 0.50
                Doc:  Integer initialization function of G1DList
```

```
- G1DList
        List size:      81
        List:           [5, 9, 6, 2, 7, 3, 8, 1, 4, 7, 8, 1, 6, 5, 4, 2, 3, 9, 3, 4, 2, 8, 9, 1, 6, 7, 5, 8, 1, 7, 9, 4,
2, 3, 5, 6, 9, 3, 5, 1, 8, 6, 4, 2, 7, 6, 2, 4, 7, 3, 5, 1, 9, 8, 2, 5, 3, 4, 6, 7, 9, 8, 1, 1, 6, 9, 5, 2, 8, 7, 4, 3,
4, 7, 8, 3, 1, 9, 5, 6, 2]
```

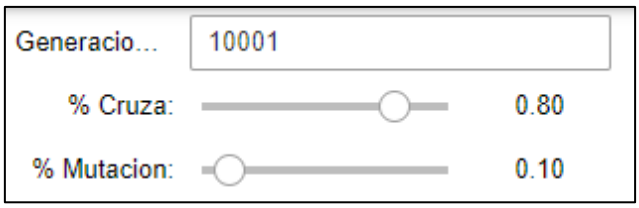
Aquí podemos ver que el sudoku se ha resuelto satisfactoriamente

```
[5, 9, 6, 2, 7, 3, 8, 1, 4,
 7, 8, 1, 6, 5, 4, 2, 3, 9,
 3, 4, 2, 8, 9, 1, 6, 7, 5,
 8, 1, 7, 9, 4, 2, 3, 5, 6,
 9, 3, 5, 1, 8, 6, 4, 2, 7,
 6, 2, 4, 7, 3, 5, 1, 9, 8,
 2, 5, 3, 4, 6, 7, 9, 8, 1,
 1, 6, 9, 5, 2, 8, 7, 4, 3,
 4, 7, 8, 3, 1, 9, 5, 6, 2]
```

Descripción de la solución

Como resultado, se obtuvo que a como aumentaban las generaciones, disminuía el porcentaje de error, en este caso, la función de aptitud. Para ello, en la gráfica se muestra tres elementos fundamentales que debemos considerar para interpretar correctamente los datos, que son: el máximo, mínimo y el promedio que se está teniendo en cada generación. Con un tiempo total de 315.142 segundos ha terminado en resolver el algoritmo.

En las 10 ejecuciones siempre llegaba a un score de 64 aproximado en cada corrida, en este caso, nunca sobrepasaba el límite de 64 en cada una de las iteraciones. En cambio, en la corrida 11, donde se modificaron los datos (cruza, mutación, numero de generaciones), que son los siguientes parámetros proporcionados:



A partir de esos parámetros, se obtuvo el siguiente resultado:

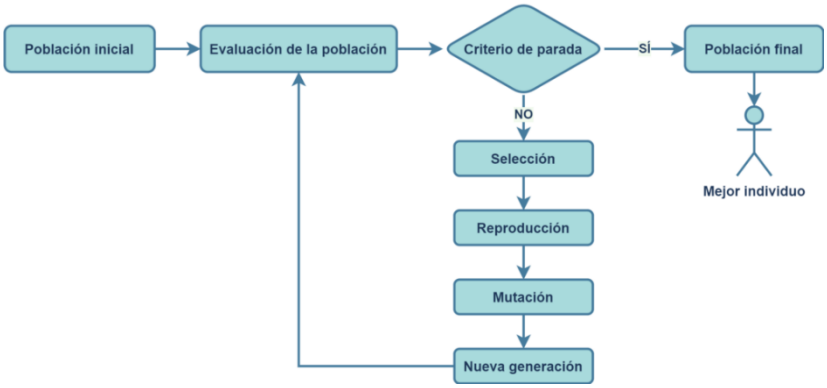
```
Gen. 10001 (100.00%): Max/Min/Avg Fitness(Raw) [161.41(153.00)/45.04(73.00)/134.51(134.51)]
Total time elapsed: 315.142 seconds.
```

```
Score:                  73.000000
Fitness:                45.038916
```

Cabe mencionar, que en la última iteración que llegó, no ha resuelto en 100% en su totalidad el sudoku.

A diferencia de las 10 corridas que se ejecutaron, se determinó que en la corrida 11, el score fue 73, quiere decir que tuvo más margen de error al ejecutarse. Además, la función de aptitud resulto con un 45.03.

Por lo tanto, cuando reducimos la cruza y aumentamos la mutación, nos pudimos dar cuenta que aumenta el margen de error.



Conclusiones

En conclusión, en realidad vivimos en una sociedad cada vez más dependiente de las nuevas tecnologías, donde las personas siempre están tratando de automatizar y hacer que nuestras actividades diarias sean lo más fáciles posible. Estos últimos entran en juego con los llamados "algoritmos universales/evolutivos", que, como hemos visto, no son cualquier método adaptativo que se pueda utilizar para resolver problemas de búsqueda y optimización. Estos algoritmos son capaces de generar soluciones a problemas reales. Para este proyecto, nuestro objetivo fue desarrollar una aplicación que pueda resolver completamente Sudoku utilizando un algoritmo general utilizando el método de selección de la rueda de la ruleta. Algunas de las cosas que pudimos descubrir mientras hacíamos este proyecto fueron que los algoritmos genéticos suelen dar resultados "no malos" en el primer caso, pero (en la mayoría de los casos) no suelen dar la mejor respuesta a la pregunta, aunque pueden dar una buena respuesta o no.

La otra cosa interesante que pudimos observar fue la utilidad de mutación, que es una variable muy interesante porque si bien puede ser útil cuando el algoritmo general se atasca, en realidad abusar de él degrada su algoritmo de búsqueda general. Casi por accidente, por esta razón (y gracias al ejemplo que hicimos) concluimos que las mutaciones deben ser lo más pequeñas posible y deben funcionar de la misma manera que en la evolución natural, es decir. poco frecuentemente. También destacamos la importancia del cruce, ya que se considera el operador genético más importante, ya que permite la combinación simultánea de buenos bloques de cromosomas de sus padres (también seleccionados como los mejores individuos). Tarde o temprano conduce a posibles soluciones. Nos dimos cuenta de que si no elegíamos buenos parámetros iniciales, el algoritmo tardaría muchas veces más que si lográramos elegir parámetros que se ajusten al problema. Otra cosa que hemos observado es que la selección de la ruleta es una opción bastante inestable porque depende en gran medida del azar, por lo que, a diferencia de otros tipos de selección (como los torneos o las tablas de clasificación), eventualmente nos damos cuenta de que a medida que disminuimos el estado físico, se vuelve cada vez más difícil para el algoritmo para seguir encontrando mejores descendientes, por lo que en este punto nuestro programa es propenso a bucles largos.

Referencias

Valencia, P. E. (1997, August). Optimización mediante algoritmos genéticos. In Anales del Instituto de Ingenieros de Chile (Vol. 109, No. 2, pp. 83-92).

Marcos; Rivero Gestal (Daniel; Rabuñal, Juan Ramón; Dorado, Julián; Pazos, Alejandro), & Gestal, M. (2010). Introducción a los algoritmos genéticos y la programación genética (p. 32). Coruña: Universidade da Coruña.

Marczyk, A. (2004). Algoritmos genéticos y computación evolutiva. Departamento de Informática, universidad de Colorado.}

Melián, M. B., Moreno, J. A., & Moreno, J. M. (2009). algoritmos genéticos. una visión práctica. Números. Revista de Didáctica de las Matemáticas, 71, 29-47.